

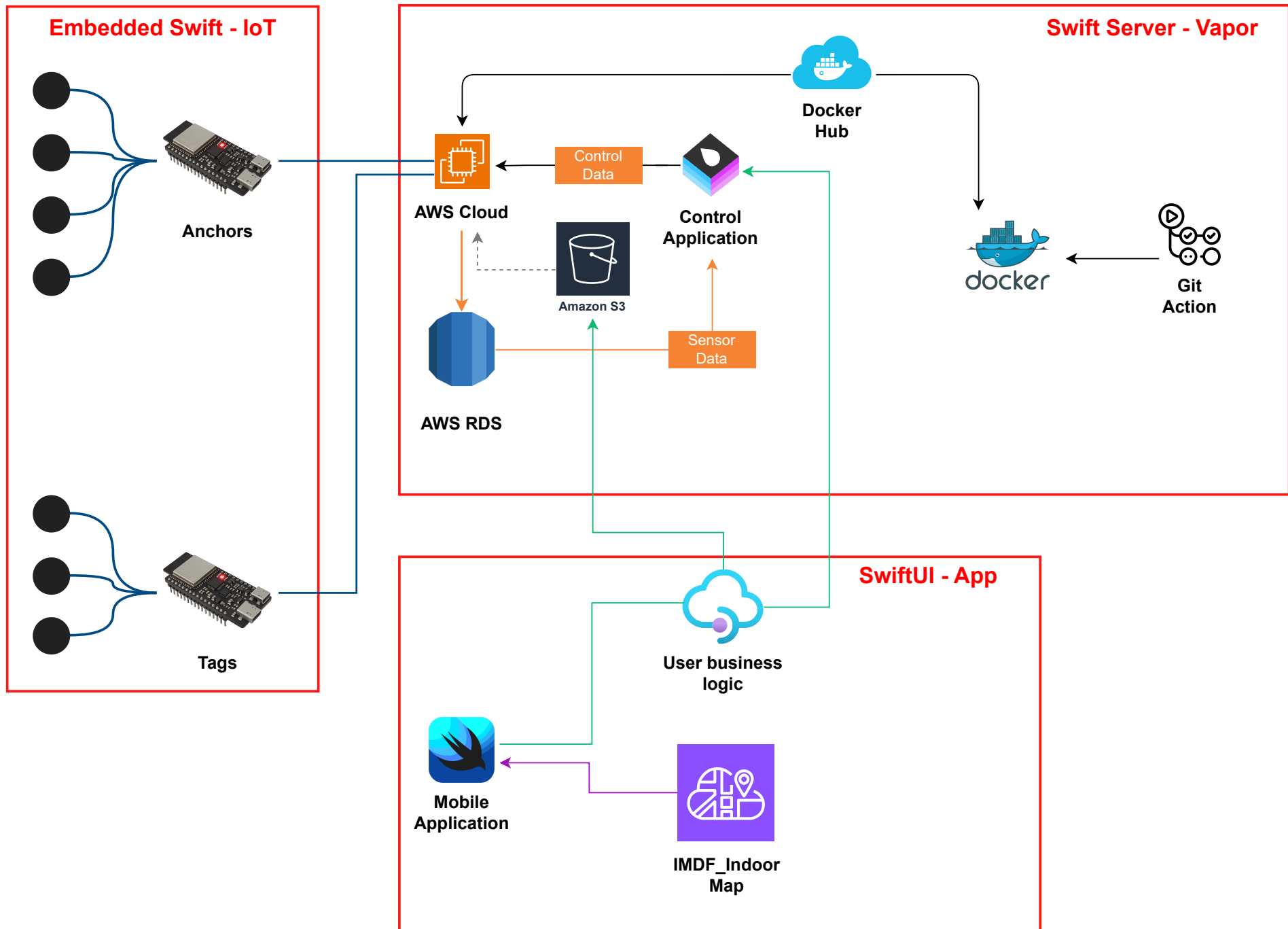
Showcase Summary



Tech

A5. Tol Story

Anne. JeOng. Kaia. Luka. One. Yuha



SwiftUI_App



Architecture 및 Combine Moya

MVVM(Model – View – ViewModel) 패턴을 기반으로 하며 Clean Architecture의 개념을 일부 도입하여 계층을 분리했다. 데이터의 흐름은 단방향으로 이루어지며, 각 계층은 명확한 역할을 가진다.

데이터 흐름은 사용자가 보는 View에서 시작된다.

View는 ViewModel의 상태 변화를 감지하여 화면을 그린다. ViewModel은 사용자의 입력을 받아 비즈니스 로직의 단위인 UseCase를 호출하고, UseCase는 실질적인 데이터를 처리를 위해 Service 계층에 요청을 보낸다. 마지막으로 Service는 Router에 정의된 명세에 따라 서버와 통신한다.

Core - 핵심 유틸리티(DIContainer, Logger)

Model - 데이터 모델(DTO, Entity)

Network - 네트워크 관련 코드

Common - 공통 네트워크 코드

Router – API 명세

Service - 실제 API 호출 로직

TokenRefresh - 토큰 갱신 로직

Network Layer Implementation

네트워크 계층은 Moya와 Combine을 결합하여 POP 방식으로 구현했다. 코드 중복을 최소화하고 타입 안정성을 보장한다.

BaseAPIService는 모든 서비스의 기반이 되는 프로토콜이다. MoyaProvider 생성, Combine Publisher 변환, JSON디코딩, 스레드 처리 등 반복되는 작업을 캡슐화한다.

```
import Foundation
import Moya
import Combine
import CombineMoya

protocol BaseAPIService {
    associatedtype Target: TargetType

    var provider: MoyaProvider<Target> { get }
    var decoder: JSONDecoder { get }
    var callbackQueue: DispatchQueue { get }
}

extension BaseAPIService {
    func request<T: Codable>(_ target: Target) -> AnyPublisher<ResponseData<T>, MoyaError> {
        provider.requestPublisher(target)
            .filterSuccessfulStatusCodes()
            .map(ResponseData<T>.self, using: decoder)
            .receive(on: callbackQueue)
            .eraseToAnyPublisher()
    }

    func requestDirect<T: Codable>(_ target: Target) -> AnyPublisher<T, MoyaError> {
        provider.requestPublisher(target)
            .filterSuccessfulStatusCodes()
            .map(T.self, using: decoder)
            .receive(on: callbackQueue)
            .eraseToAnyPublisher()
    }
}
```

Network Advantages

아키텍처는 BaseAPIService의 Extension에 요청, 검증, 디코딩, 스레드 전환 등 반복되는 공통 로직을 집약하여, 실제 서비스 구현 시 request(target) 단 한줄만으로 API를 추가할 수 있게 개발 생산성을 극대화했습니다. 제네릭을 활용해 컴파일 시점에 타입 안전성을 보장하는 한편,

FilterSuccessfulStatusCodes()를 통해 모든 API 요청이 동일한 성공 및 실패 기준을 따르도록 하여 시스템 전반의 일관성을 확보했다.

Memory & CPU Stability

Combine의 생명주기 관리와 Moya의 추상화를 통해 시스템 리소스를 매우 효율적으로 사용한다.

```
// MARK: - Patient Method
/// 환자 목록 조회 API 호출
func listPatient() {
    isLoading = true

    container.usecaseProvider.patientUseCase.executeGetList()
        .validateResult()
        .sink { [weak self] completion in
            guard let self else { return }
            defer { self.isLoading = false }

            switch completion {
            case .finished:
                Logger.logDebug("환자 목록 조회", "요청 완료 (finished)")
            case .failure(let error):
                Logger.logError("환자 목록 조회", "실패: \(error.localizedDescription)")
            }
        }
        .receiveValue { [weak self] result in
            guard let self else { return }
            self.patientsData = result
        }
        .store(in: &cancellables)
}
```

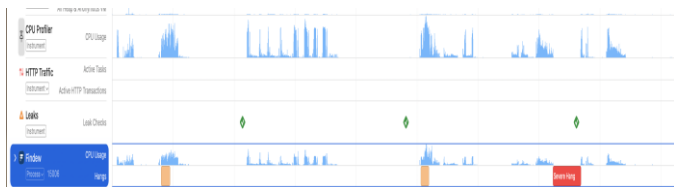
우선 메모리 측면에서는 구조적으로 메모리 누수를 차단했다. Combine의 cancellables를 활용하여 구독을 관리함으로써, ViewModel이 해제되는 즉시 연결된 네트워크 요청과 구독이 자동으로 취소되도록 설계했다. 덕분에 사용자가 화면을 빠르게 이탈하더라도 불필요한 네트워크 리소스가 메모리에 남지 않으며, 클로저 내부에서 [weak self]를 사용하는 패턴과 결합하여 순환 참조로 인한 좀비 객체 생성 위험을 원천적으로 방지했다.



또한 CPU 연산 효율성을 높여 UI 성능을 최적화했다. 비용이 높은 JSON 파싱 작업은 메인 스레

드가 네트워크 응답이 도착한 백그라운드 큐에서 .map 연산자를 통해 먼저 처리된다. 무거운 데이터가 가공 작업을 백그라운드로 분리함으로써 대용량 데이터 처리 시에도 화면 버벅임이 발생하지 않도록 했다. 이후 데이터 준비가 완료된 시점에만 receive를 통해 메인 스레드로 진입하게 하여, 불필요한 문맥 교환 비용을 줄이고 UI 반응성을 극대화했다.

마지막으로 리소스 재사용을 통해 오버헤드를 최소화했다. APIManager.shared 싱글톤을 통해 무거운 객체인 Session과 JSONDecoder 인스턴스를 공유하고 재사용합니다. API 요청이 빈번하게 발생하더라도 매번 객체를 생성하고 파괴하는 비용을 절감하여, 앱의 메모리 사용량을 일정하고 안정적인 수준으로 유지할 수 있다.



IMDF 기반 실내 지도 렌더링 시스템

해당 시스템은 Apple의 IMDF 표준을 준수하여 실내 공간 정보를 시각화하는 모듈이다. 복잡한 지리 공간 데이터를 모바일 환경에서 효율적으로 표출하기 위해, SwiftUI의 선언형 MapKit 프레임워크 결합했다. 특히 고비용 렌더링 객체인 지도의 성능 최적화를 위해 상태 비교 기반의 뷰 갱신 전략을 채택했다.

IndoorMapViewModel과 IndoorMapView로 처리 로직과 프레젠테이션 계층을 분리했다.

IndoorMapViewModel은 비즈니스 로직을 담당하며, IMDF 파일의 디코딩, 층별 데이터 필터링, MKOverlay 객체로의 변환을 수행한다.

IndoorMapView는 사용자 인터페이스를 담당하며, ViewModel이 발행하는 오버레이 데이터를 관찰하여 지도에 폴리곤을 렌더링한다.

최적화 전략

지도 뷰는 초기화 및 렌더링 시 시스템 리소스 (CPU/GPU) 소모가 큰 컴포넌트이다. SwiftUI의 특성상 상위 뷰의 상태 변화가 하위 뷰의 Re-evaluation을 유발할 수 있는데, 지도 뷰의 불필요한 재생성과 프레임 드랍의 원인이 된다. 이를 해결하기 위해 IndoorMapView는 Equatable 프로토콜을 채택하여 명시적인 뷰 갱신 규칙을 정의했다.

```
// MARK: - Equatable
static func == (lhs: Self, rhs: Self) -> Bool {
    return lhs.floor == rhs.floor
}
```

이전 뷰 인스턴스와 현재 뷰 인스턴스의 floor 프로퍼티를 비교한다. 두 값이 동일한 경우, SwiftUI의 Diffing 엔진은 해당 뷰의 내부 상태가 변경되지 않았다고 판단하여 body 프로퍼티의 재호출을 생략한다. 사용자가 동일한 층을 탐색하는 동안 지도가 깜빡이거나 불필요하게 리로드되는 현상을 원천적으로 차단하여 Rendering Stability를 보장한다.



공간 데이터의 로드 및 오버레이 변환

IMDF 데이터는 GeoJSON 형식을 기반으로 하며, 본 시스템은 이를 시각화 가능한 객체로 변환하기 위해 3단계 파이프라인을 수행한다.

[1단계 - Decoding]

IMDFDecoder를 통해 앱 번들 내의 IMDF 리소스를 비동기적으로 파싱하여 Venue 객체 모델로 변환한다.

[2단계 - Hierarchical Filtering]

Venue.levelsByOrdinal 딕셔너리를 참조하여, 현재 요청된 층 Ordinal에 속한 유닛, 시설, 편의 시설 등의 지형지물만을 추출한다.

[3단계 - Overlay Instantiation]

추출된 각 Feature의 기하학 정보를 MKPolygon, MKPolyline 등의 MKOverlay 객체로 매핑한다. 이때 OverlayData 구조체를 통해 각 오버레이에 대응하는 스타일 정보를 캡슐화하여 뷰 계층으로 전달한다.

공간 데이터의 로드 및 오버레이 변환

실내지도는 일반 지도와 달리 특정 건물의 좁은 영역을 정밀하게 보여주어야 한다. 따라서 데이터 로드가 완료된 시점에 사용자의 시점을 ROI로 자동 이동시키는 로직이 필수적이다. 해당 시스템은 건물의 중심 좌표 또는 건물의 경계 사각형을 계산한다. 이를 기반으로 MKCoordinateRegion을 생성하여 카메라를 초기화함으로써 앱 실행 시 사용자가 별도의 조작 없이도 건물의 전체 형상을 즉시 인지할 수 있도록 UX 최적화했다.

Embedded Swift_IoT



ESP32-C6과 Embedded Swift

우리 프로젝트는 애플이 주도하고 있는 Embedded Swift를 활용하기 위해 ESP32-C6을 주요 플랫폼으로 선택했다. ESP32-C6은 RISC-V 아키텍처 기반 칩으로 Swift 컴파일러가 직접 이해할 수 있는 명령어 집합을 제공한다. Swift 컴파일러는 코드를 먼저 LLVM IR이라는 중간 표현으로 변환한 뒤, 이를 RISC-V 기계어로 최적화하여 변환한다. 이런 과정 덕분에 복잡한 변환 절차 없이도 Swift의 안전성과 표현성을 그대로 활용할 수 있다는 점이 큰 장점이다.

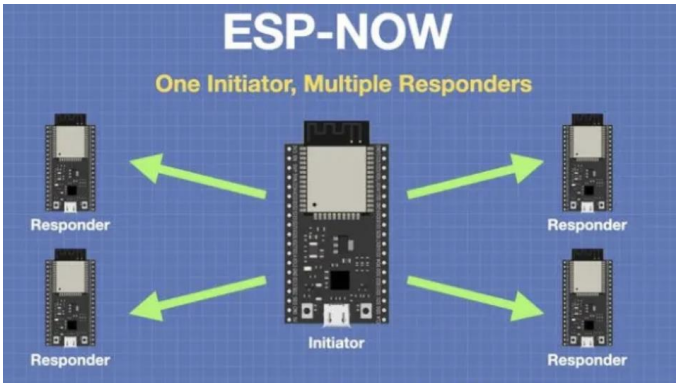
또한 ESP32-C6은 Wi-Fi 6와 Matter 규격을 지원하여 최신 IoT 표준과 높은 호환성을 보인다. 특히 Wi-Fi 6의 향상된 전력 효율은 배터리 기반 장치인 비콘의 사용 시간을 크게 늘려 준다.

프로젝트에서는 ESP32-C6의 다양한 기능 중 ESP-NOW와 FTM(Fine Time Measurement) 기능을 핵심적으로 활용하였다. ESP-NOW는 장치 간 고속 정보 전달을 담당하며, FTM은 거리 측정과 위치 추정에 사용된다. Wi-Fi 6 기반의 정교한 RF 성능 덕분에 FTM의 측정 정밀도도 함께 향상된다.

ESP-NOW : 네트워크 없이 동작하는 초저지연 P2P 통신

ESP-NOW는 Espressif가 개발한 독자적인 무선 통신 프로토콜로, IEEE 802.11 표준을 기반으로 하면서도 기존 Wi-Fi에서 요구되는 네트워크 계층 과정을 생략한다. AP에 접속할 필요 없이 기기 간 P@P 방식으로 직접 데이터를 주고받을 수 있어 매우 낮은 지연 시간과 높은 안정성을 제공한다.

일반적인 Wi-Fi 통신 연결, 인증, DHCP, 라우팅 등 여러 과정을 거쳐야 하지만, ESP-NOW는 이러한 절차 없이 MAC 주소만 알고 있으면 즉시 통신이 가능하다. 덕분에 ms 단위의 빠른 전송이 가능하며, 네트워크 환경이 불안정하거나 오프라인에서도 문제 없이 작동한다.



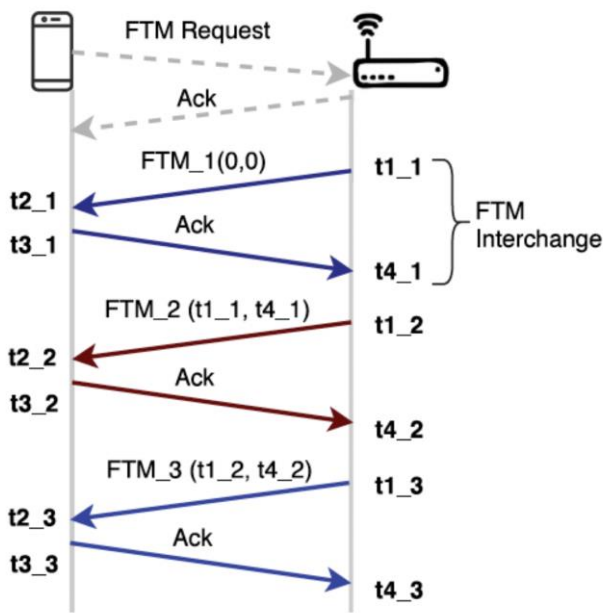
우리 프로젝트에서는 ESP-NOW는 앵커와 태그 간의 정보 교환에 활용되었다. 게이트웨이는 자신의 층 정보를 브로드캐스트 방식으로 주변 모든 비콘에게 전송한다. 비콘은 이를 수신한 뒤 FTM으로 계산한 거리 정보와 함께 데이터 패킹을 구성한다. 이후 가장 강한 RSSI를 가진 게이트웨이

를 선택해 해당 데이터를 ESP-NOW로 전송하고, 게이트웨이가 이를 서버에 전달하여 앱에서 활용할 수 있게 된다.

FTM – Time of Flight 기반 고정밀 거리 측정

앵커의 정확한 위치를 파악하기 위해 프로젝트에서는 FTM(Fine Time Measurement) 기술을 도입했다. FTM은 IEEE 802.11mc 표준에 정의된 방식으로, 기존의 RSSI 처럼 신호 세기를 이용하는 것이 아니라, Time of Flight를 기반으로 거리를 측정한다. 이 방식은 환경 변화나 장애물의 영향을 덜 받기 때문에 훨씬 정밀한 결과를 얻을 수 있다.

FTM은 RTT(Round Trip Time) 방식으로 동작한다. 앵커(FTM-Initiator)과 게이트웨이(FTM-Responder)가 FTM 프레임을 교환하며 ns(나노초) 단위의 타임스탬프를 기록하고, 이 값을 이용해 전파가 이동한 시간을 계산한다. 하지만 실내 환경에서는 다중 경로 간섭과 노이즈로 인해 오차가 발생할 수 있기 때문에, 우리는 추가적으로 킬만 필터를 적용하여 측정값을 보정했다.



보정 과정은 이전 거리값을 기반으로 현재 위치를 예측하고, 새로 들어온 FTM 측정값과 비교해 신뢰도를 평가한 뒤 두 값을 적절히 결합하는 방식으로 작동한다. 여러 게이트웨이로부터 얻은 거리 정보를 활용하면 삼변측량으로 더욱 정확한 위치를 산출할 수 있다.

단어 개념 정리

[RISC-V]

오픈소스로 공개된 소프트웨어의 명령을 이해하고 실행하는 규칙을 정의한 명령어 집합 구조다. 구조가 단순하고 효율적이라 전력 소모가 적으며, 최근 애플 등 빅테크 기업들이 차세대 임베디드 표준으로 주목하고 있다.

[Wi-Fi 6]

데이터 전송 속도뿐만 아니라 다중 접속 효율성과 전력 소모 절감에 최적화된 6세대 무선 통신 표준이다. ESP32-C6은 이 표준을 지원함으로써 얻는 이점은 배터리 수명을 비약적으로 연장할 수 있고, 하나의 통신 채널을 더 작은 단위로 분리하여 트래픽없이 효율적으로 처리할 수 있게 된다.

[Matter]

애플의 Home IoT 전략과 Embedded Swift의 효율성을 연결하는 중요한 기술이다. 애플, 구글, 아마존 등 글로벌 기업들이 연합하여 만든 개방형 스마트 홈 연동 표준이다. 특히 애플에서는 Embedded Swift를 통해 C++ 기반의 복잡하고 불안정한 Matter 펌웨어 개발 과정을 안전하고 직관적인 Swift-based 방식으로 혁신하고 있다. 결과적으로 애플 생태계와 호환되는 IoT 기기가 더 활발하게 시장에 나오도록 돕는다.

[LLVM/IR]

애플이 Swift 언어를 개발할 때 근간으로 삼은 모듈식 컴파일러 인프라, 프로그래밍 언어를 기계어로 번역해주는 엔진으로 IoT 프로젝트에서 핵심 역할을 한다. LLVM이 Swift 코드를 중간언어인 IR로 변환한 뒤, 이를 타겟 하드웨어인 RISC-V 명령어로 최적화하여 재변환한다. 이 중간 과정으로 인해 별도의 복잡한 변환 과정 없이도 ESP32-C6에서 Swift의 안전성과 성능을 활용할 수 있게 된다.

[IEEE 802.11]

흔히 Wi-Fi라고 부르는 기술의 공식적인 국제 표준 규격이다. 이 프로토콜에서는 네트워크 7계층 중 물리계층과 매체 접근 제어 계층을 정의한다. 프로젝트 내부에서 ESP-NOW와 FTM에 적용된다.

[RSSI]

수신된 무선 신호의 전력이 얼마나 강한지를 나타내는 적도이다. 음의 dbm 단위로 표현되며, 0에 가까울수록 신호가 강함을 의미한다. 일반적으로 위치 추적 시스템에서 RSSI 값을 활용하는데 RSSI는 환경 변화에 민감하고 장애물에 취약하다. 따라서 우리 프로젝트에서는 이 한계를 뛰어넘고자 FTM 기술을 도입했다.

[브로드캐스트]

네트워크에 연결된 모든 기기에서 동시에 데이터를 전송하는 one to All 통신 방식이다. 실제 구현할 때는 Mac 주소를 FF:FF:FFFF:FF:FF로 작성하여

사용한다. 우리의 프로젝트에서는 게이트웨이가 자신의 총 데이터를 태그에게 전달할 때, 해당 공간에 존재하는 모든 태그에 전달해야 함으로 브로드캐스트 방식을 채택했다.

[FTM]

FTM은 FTM-Initiator와 FTM-Responder 역할로 나뉘어 측정한다. 우리의 프로젝트에는 태그가 FTM을 측정하는 FTM-Initiator 모드로, 앵커가 FTM을 응답하는 FTM-Responder 모드로 설정했다. 추가적으로 ESP32-C6이 FTM-Initiator를 지원하려면 칩 리비전 버전이 0.2 이상이 되어야 하고, FTM-Initiator는 STA 모드에서만, FTM-Responder는 AP 모드에서만 지원한다.

[AP/STA 모드]

각각 무선 네트워크에서 기기가 수행하는 역할을 정의하는 용어다. STA는 일반적으로 우리가 사용하는 스마트폰이나 노트북처럼, 와이파이 신호를 받아 접속하는 클라이언트 모드이다. AP는 공유기처럼 와이파이 신호를 송출하여 다른 기기들이 접속할 수 있게 해주는 호스트 모드다. 추가적으로 ESP32-C6은 AP, STA 모드 모두 지원하며, 하이브리드 방식인 APSTA 모드도 지원한다.

Swift Server_Vapor



Swift는 Apple의 생태계를 넘어 서버 측 개발 영역으로 활용 범위를 확장했다. 서버측 Swift는 타입 안정성, 고성능, iOS/macOS 개발자와의 언어 통일성이라는 이점을 제공한다.

서버 측 생태계에서 Vapor는 가장 대표적이고 활발하게 개발되는 비동기 Event-Driven 웹 프레임워크이다. Vapor의 핵심 기술 및 아키텍처를 분석하고 기존의 주류 웹 프레임워크와 비교하여 성능적 특징과 실질적인 채택 가치를 평가하는 게 목표다.

Vapor 프레임워크의 구조적 특징

Vapor는 Apple에서 개발한 SwiftNIO를 기반으로 하며 논블로킹, 이벤트 기반 아키텍처를 사용한다. 높은 동시성, 확장성을 가능하게 하는 핵심 요소이다.

[Swift NIO]

네트워크 애플리케이션 개발을 위한 저수준 도구 키트

Vapor의 고성능 I/O 처리를 위한 기반[Swift Concurrency]

Swift 5.5부터 도입된 비동기 프로그래밍 모델을 활용하여 명확하고 쉽게 작성 가능

[모듈성]

Vapor는 코어 기능 외에 Fluent ORM, Leaf, JWT 등 모듈화된 패키지를 제공하여 개발자가 필요한 기능만 선택적으로 통합할 수 있다.

Vapor 아키텍처 및 핵심 구성 요소

1. Request Handling Flow

Vapor의 요청 처리는 Middleware -> Routing -> Controller/Handler 단계를 거친다

[미들웨어]

요청이 라우트 핸들러에 도달하기 전/후에 실행되는 계층

Authentication, Logging, 세션 관리, 오류 처리 등

을 모듈화하여 처리한다

[라우팅]

HTTP 메소드와 URL 경로를 특정 핸들러에 연결하는 역할을 한다.

RouteGroup을 통해 코드의 조직화 및 유지보수성을 높인다

2. Fluent ORM

Fluent는 Vapor의 공식 ORM 솔루션으로 Swift의 타입 안전성을 활용하여 데이터베이스 조작을 SQL 없이 네이티브 Swift 코드로 가능하게 한다.

[지원 DB]

PostgreSQL, MySQL, MongoDB 등 다양한 DB Driver 지원

[Migration]

데이터베이스 스키마 변경을 관리하는 기능을 제공하여 버전 관리를 용이하게 한다.

3. Dependency Injection and Service Container

Vapor의 아키텍처는 Service Container 패턴을 핵심적으로 활용한다. 컨테이너는 애플리케이션의 설정, 로깅, 데이터베이스 연결, 커스텀 서비스 등 모든 구성 요소를 등록하고 관리하는 역할을 한다

[Inversion Of Control]

개발자가 객체의 생성과 생명 주기를 직접 제어하는 대신, 컨테이너에 위임한다. 컴포넌트 간의 결합도를 낮추고 모듈성을 극대화하여 유지보수가 용이하다.

[Application Object]

이 객체가 서비스 컨테이너의 역할을 수행하며, Request 객체를 통해 등록된 모든 서비스에 접근할 수 있게 한다.

4. Authentication And Authorization

Vapor는 웹 애플리케이션 및 API 보안을 위해 포괄적인 인증 기능을 제공한다

[인증 미들웨어]

요청 처리 과정에서 사용자의 신원을 확인하고 Request 객체에 인증된 사용자 정보를 추가하는 미들웨어를 손쉽게 정의할 수 있다

[세션 기반 인증]

전통적인 웹 애플리케이션을 위해 Session을 통한 사용자 상태 관리를 지원한다.

[JWT]

RESTful API 환경에서 널리 사용되는 JSON Web Token 처리를 위한 공식 패키지를 지원하여 토큰 생성, 서명, 검증을 안전하게 처리할 수 있다. 서버의 상태를 유지할 필요가 없어 높은 확장성에 기여한다

[인증 및 권한 부여 프로토콜]

사용자의 역할을 기반으로 특정 리소스에 대한 접근 권한을 제어하는 Guard와 같은 기능을 통해 권한 부여 로직을 간결하게 구현할 수 있다.

4. Vapor 모델의 핵심 구성 요소

데이터 모델은 final class로 정의되며 Model 프로

토콜을 준수해야 합니다. Model 프로토콜에 따라
데이터베이스 테이블의 이름을 지정하는

순화한다.

```
static let schema =  
"table_name"
```

정의해야 합니다. Fluent가 데이터베이스 쿼리 결
과를 기반으로 모델을 초기화할 수 있도록 빈
init() 초기화 함수 또한 필수적으로 작성해야 합
니다.

기본적인 구조 위에, 모델의 속성들은 속성 래퍼
를 사용하여 데이터베이스의 각 컬럼에 매핑된다.
고유 식별자 @ID 속성 래퍼를 통해 정의되며,
Fluent가 데이터베이스에서 해당 모델을 찾는 데
사용된다. 일반적인 데이터 필드는 @Field 또는
@OptionalField 속성 래퍼를 사용하여 나타내며
이때 Key 매개변수는 데이터베이스의 실제 컬럼
이름을 지정한다. 속성 래퍼의 사용은 컴파일러
주도의 타입 안전성을 확보하며, Fluent가 전체 모
델이 아닌 개별 필드를 효율적으로 업데이트하거
나 선택적으로 검색할 수 있도록 한다. Vapor 모
델이 클라이언트 - 서버 통신에서 JSON 데이터
를 처리할 수 있도록 Content 프로토콜을 확장하
여 채택한다. Content는 Swift의 Codable을 래핑
한 형태로 API 요청 본문을 모델로 디코딩하거나
응답 시 모델을 JSON으로 인코딩하는 과정을 단

관계 모델링과 데이터 무결성 확보

데이터 간의 복잡한 연결은 Fluent 관계 모델링 기능을 통해 명확하게 정의된다

Parent - Child Relation은 One - to - Many 관계를 모델링한다. 자식 모델은 @Parent 속성 래퍼를 사용하여 부모 모델의 ID 외래 키로 참조하며, 부모 모델은 @Childre 속성 래퍼를 사용하여 해당 부모에 속한 모든 자식 모델의 컬렉션을 나타낸다. 만약 관계가 로드되지 않은 상태에서 부모 또는 자식 모델에 접근을 시도하면 에러가 발생하므로, 일반적으로 `$user.get on db` 와 같이 속성 래퍼의 투영 값을 사용하여 데이터베이스에 쿼리를 수행해야 한다.

Many - to - Many 관계인 형제 관계는 두 모델외에 이 둘을 연결하는 별도의 피벗 모델을 사용하여 구현된다. 이 피벗 모델은 관계를 맺는 두 모델에 대한 @Parent 참조를 포함하여 원래의 두 모델에는 @Siblings 속성 래퍼를 사용하여 피벗 모델의 타입과 각 모델에 대한 키 경로를 지정하여 관계를 설정한다.

아래 코드는 Model의 예시로 사용한 코드이다. 환자 정보를 표현한 Model이다

```
import Vapor
import Fluent
/// 환자 테이블
```

```
final class Patient: Model, Content, @unchecked
Sendable {
    static let schema: String =
SchemaValue.patients

    @ID(key: .id)
    var id: UUID?

    @Parent(key: "hospital_id")
    var hospital: HospitalAccount

    @Field(key: "name")
    var name: String

    @Field(key: "ward")
    var ward: String

    @Field(key: "bed")
    var bed: Int

    @Parent(key: "department_id")
    var department: Department

    @OptionalParent(key: "device_id")
    var device: Device?

    @OptionalField(key: "memo")
    var memo: String?

    @Timestamp(key: "created_at", on: .create)
    var createdAt: Date?

    @Timestamp(key: "updated_at", on: .update)
    var updatedAt: Date?

    init() {}
```

```

init(
    id: UUID? = nil,
    hospitalId: UUID,
    name: String,
    ward: String,
    bed: Int,
    departmentId: UUID,
    deviceId: UUID? = nil,
    memo: String? = nil
) {
    self.id = id
    self.$hospital.id = hospitalId
    self.name = name
    self.ward = ward
    self.bed = bed
    self.$department.id = departmentId
    self.$device.id = deviceId
    self.memo = memo
}
}

```

이러한 모델 정의가 실제 프로덕션 환경에서 안정적으로 작동하기 위해서 Migration 과정이 필수이다. 마이그레이션은 데이터베이스의 스키마를 생성하거나, 기존 테이블에 새 필드를 추가한다. 또는 카테고리 이름의 고유성과 같은 외래 키 제약 조건을 추가하여 데이터 무결성을 보장하는데 사용된다. 마이그레이션 구조체는 `prepare(on:)` 함수로 변경 사항을 적용하고, `revert(on:)` 함수로 이를 되돌리는 기능을 정의한다.

고급 데이터 관리 및 모델 라이프사이클

Fluent는 모델의 데이터 관리를 돕는 여러 고급 기능을 제공한다. `TimeStamp` 기능을 모델에 구현하면 `CreateAt`, `UpdatedAt` 속성을 통해 모델이 생성되거나 업데이트 될 때 자동으로 시간이 기록된다. `Soft Delete` 기능을 활성화하면 `DeleteAt` 필드에 삭제 시각을 기록하고 실제 데이터는 데이터베이스에서 제거되지 않도록 함으로써 데이터 보존 요구 사항을 충족 시킬 수 있다.

이 외에도 모델의 특정 단계에서 추가적인 로직을 실행해야 할 경우 "라이프사이클 후크"를 사용할 수 있다. `WillCreate` 후크를 사용하여 모델이 생성되기 전에 특정 필드의 고유성을 확인하고 유효성 검사 오류를 발생시켜 데이터베이스의 제약 조건 오류 대신 사용자 친화적인 오류 메시지를 클라이언트에게 전달할 수 있다.

Fluent 모델 정의는 서버 애플리케이션의 데이터 기반을 형성하는 핵심적인 단계이며 Swift의 언어적 특성과 통합된 ORM을 통해 안정적이고 유지보수가 용이한 백엔드 시스템을 구축할 수 있다. 모델 정의가 얼마나 견고하게 이루어졌는지에 따라 위에 쌓아 올릴 API와 웹 앱의 안정성이 결정된다고 볼 수 있다. 해당 코드는 위 `Model`을 `Migration`을 한 코드이다.

```

import Fluent

struct CreatePatient: AsyncMigration {
    func prepare(on database: any Database)
    async throws {
        try await
        database.schema(SchemaValue.patients)
            .id()
            .field(IdKeyField.hospitalId, .uuid, .required,
            .references(SchemaValue.hospitalAccount,
            "id", onDelete: .cascade))
            .field(PatientField.name, .string, .required)
            .field(PatientField.ward, .string, .required)
            .field(PatientField.bed, .int, .required)
            .field(IdKeyField.departmentId, .uuid,
            .required, .references(SchemaValue.departments,
            "id", onDelete: .restrict))
            .field(IdKeyField.deviceId, .uuid, .references(SchemaValue.devices,
            "id",
            onDelete: .setNull))
            .field(PatientField.memo, .string)
            .field(CommonField.createdAt, .datetime)
            .field(CommonField.updatedAt, .datetime)
            .unique(on: IdKeyField.hospitalId,
            PatientField.ward, PatientField.bed)
            .create()
    }

    func revert(on database: any Database) async
    throws {
        try await
    }
}

```

```

database.schema(SchemaValue.patients).delete()
    }
}

```

Middleware, Service, Controller

Vapor 프레임워크에서 미들웨어, 서비스, 컨트롤러는 요청 처리 파이프라인을 구성하는 핵심적인 세 가지 요소이며, 유기적으로 연결되어 클라이언트 요청을 처리하고 응답을 생성한다.

클라이언트로부터 HTTP 요청이 서버에 도달하면, 가장 먼저 미들웨어 인스턴스들을 통과한다. 미들웨어는 라우터와 클라이언트 사이에 위치하며 라우터가 요청을 처리하기 전에 요청을 가로채거나 라우터가 응답을 생성한 후에 응답을 변형하는 역할을 수행한다. 미들웨어의 주요 기능은 애플리케이션 전반에 걸친 cross-cutting concerns를 처리하는 것이다.

[Session Middleware]

웹사이트의 상태를 유지하기 위해 세션 ID가 담긴 쿠키를 읽고 세션에 사용자 정보가 있다면 요청에 인증된 사용자를 추가한다.

BasicAuthMiddleware나 TokenAuthMiddleware 같은 인증 미들웨어는 API 요청에 포함된 자격 증명을 확인하여 유효하지 않은 경우 401 Unauthorized 에러를 발생 시킬 수 있다.

[ErrorMiddleware]

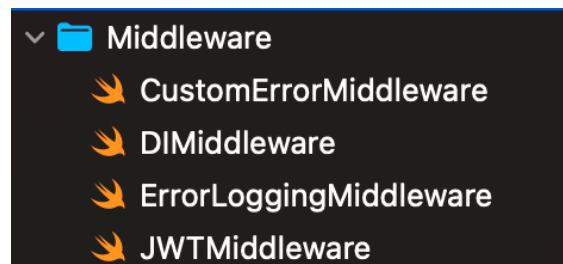
애플리케이션 내에서 발생하는 동기적 및 비동기적 Swift 오류를 가로채서 클라이언트에게 적합한 HTTP 응답으로 변환해주는 역할을 한다. 프로덕션 모드에서는 민감한 정보 노출을 막기 위해 모

든 오류를 불투명한 500 응답으로 변환한다.

[LogMiddleware]

요청 메서드, 경로, 응답 시간 등을 기록하거나 FileMiddleware를 사용하여 /Public 디렉터리에 있는 정적 파일을 서빙할 수 있다.

미들웨어는 체인 형태로 동작하며, 요청이 미들웨어 체인을 따라 내려가 최종적으로 라우터에 도달하고 응답은 다시 체인을 따라 역순으로 클라이언트에게 전달된다. 미들웨어는 경로 그룹에 적용하여 특정 경로만 보호할 수도 있다.



서비스/DI: 자원 공급 및 관리

미들웨어나 컨트롤러와 같은 주요 컴포넌트가 로직을 수행하려면 로거, 데이터베이스 커넥션, 캐시 저장소 등 다양한 자원이 필요하다. Vapor는 Container 시스템을 통해 이러한 종속성을 관리하고 주입한다. Vapor는 Environment 속성을 활용하여 환경에 따라 서비스를 동적으로 구성할 수 있는 강력한 유연성을 제공한다.

데이터베이스 설정 시 환경 변수를 통해 프로덕션 환경에서는 PostgreSQL 서버에 연결하고 개발 환경에서는 로컬 데이터베이스를 사용하도록 설정할 수 있다. KeyedCache와 같이 여러 구현체가 존재하는 프로토콜의 경우 config.preper 지시자를 사용하여 애플리케이션이 어떤 구현체를 사용할지 명시적으로 지정하여 서비스의 모호성을 해결할 수 있다. 서비스 관리 체계는 애플리케이션의 핵심 로직을 자원 제공 방식과 분리하여 프로덕션 배포 및 테스트를 용이하게 한다. 아래 코드와 같이 서비스와 DI를 작성할 수 있다.

```
import Vapor
/// Dependency Injection Container
///
/// Request 기반으로 Service를 생성합니다.
struct DIContainer {

    let request: Request

    init(request: Request) {
```

```
        self.request = request
    }

    // MARK: - Services

    /// AuthService 생성
    func makeAuthService(request: Request) ->
AuthService {
        return AuthService(
            database: request.db,
            app: request.application
        )
    }

    /// RoomService 생성
    func makeRoomService(request: Request) ->
RoomService {
        return RoomService(database: request.db)
    }

    /// DeviceService 생성
    func makeDeviceService(request: Request) ->
DeviceService {
        return DeviceService(database: request.db)
    }

    /// PatientService 생성
    func makePatientService(request: Request) ->
PatientService {
        let deviceService =
makeDeviceService(request: request)
        return PatientService(database:
request.db, deviceService: deviceService)
    }

    /// DepartmentService 생성
```

```

func makeDepartmentService(request:
Request) -> DepartmentService {
    return DepartmentService(database:
request.db)
}

/// AnchorService 생성
func makeAnchorService(request: Request) ->
AnchorService {
    return AnchorService(database:
request.db)
}

/// ReportService 생성
func makeReportService(request: Request) ->
ReportService {
    return ReportService(database:
request.db)
}

func makeLocationService(request: Request) -
> LocationService {
    let deviceService =
makeDeviceService(request: request)
    let anchorService =
makeAnchorService(request: request)
    return LocationService(database:
request.db, deviceService: deviceService,
anchorService: anchorService)
}

func makeS3Service(request: Request) ->
S3Service {
    return S3Service(app: request.application)
}

/// DashboardService 생성

```

```

func makeDashboardService(request: Request)
-> DashboardService {
    return DashboardService(database:
request.db)
}

```

컨트롤러: 비즈니스 로직 실행 및 응답

생성

미들웨어에 의해 보안 및 유효성 검사가 완료되고, 서비스 컨테이너를 통해 필요한 자원이 준비되면, 요청은 파이프라인의 최종 단계인 컨트롤러로 전달되어 핵심 비즈니스 로직이 실행됩니다.

컨트롤러는 프로젝트의 라우트 핸들러를 `route.swift` 파일 하나에 모아두는 대신 코드를 구조화하여 관리하기 위해 사용된다. 컨트롤러는 `RouteCollection` 프로토콜을 준수해야 한다. `boot(routes:)` 함수를 구현하여 자신이 담당하는 모든 경로를 등록한다. 컨트롤러 내의 각 라우트 핸들러는 수신된 `Request` 객체를 분석하, `Fluent ORM`을 활용하여 데이터 모델에 대한 `CRUD(Create, Retrieve, Update, Delete)` 데이터베이스 작업을 수행한다.

컨트롤러의 주요 임무는 요청된 로직을 수행한 후 클라이언트에게 반환할 최종 응답을 구성하는 것이다. 아래와 같이 코드를 작성하여 완성할 수 있다.

명세화

컨트롤러가 비즈니스 로직을 구현하여 API 기능을 완성했다면 다음 단계는 이 기능을 외부에 명확하고 표준화된 방식으로 문서화하고 시각화하는 것이다. 구축된 API가 어떤 경로를 지원하고, 어떤 데이터 형식을 요구하는지에 대한 상세한 명세는 협업에 필수적이다. OAS, 그리고 시각적으로 보여주는 `Swagger UI`를 활용하는 과정이 필요하다

`OpenAPI` 정의는 `YAML` 또는 `JSON` 형태의 문서로 작성되며, API의 기능, 매개변수, 응답 그리고 보안 요건을 상세하게 기술하는 공식적인 데이터 정의가 된다.

1. Vapor 환경에서의 AP 명세 생성 방법

`Vapor` 생태계에서 API 명세를 생성하는 두 가지 주요 접근 방식이 존재하며, 각각 코드 우선 방식과 명세 우선 방식에 해당한다

[코드 우선 방식]

`Vapor` `Swift` 코드로부터 `OpenAPI` 명세를 자동으로 추출하고 생성하는 방식이다.

`VaporToOpenAPI` 라이브러리는 `Vapor` 기반 프로젝트를 위해 특별히 설계되었으며 다른 언어의 `Swagger` 문서 생성 방식과 유사하게 느껴져 사

용성이 편리하다. 이 도구를 사용하면 Vapor의 애플리케이션의 라우트 빌더에 `.openAPI(info:)` 메소드를 호출하여 API 메타데이터를 포함한 `OpenAPIObject`를 생성하고 특정 경로에 등록하여 외부에 제공할 수 있다.

[명세 우선 방식]

Apple의 `Swift-openapi-generator` 도구는 미리 작성된 OpenAPI 문서를 입력받아 이 명세에 기반하여 요청을 처리하는 서버 스텝 또는 클라이언트 코드를 자동으로 생성한다. Vapor 프레임워크와의 연동은 `Swift-openapi-vapor` 패키지를 통해 이루어지며, 이 패키지는 생성된 코드를 Vapor 애플리케이션의 라우팅 체계에 바인딩하는 역할을 담당한다. 미리 정의된 `APIProtocol`을 준수하는 핸들러 타입의 인스턴스를 생성하고 `VaporTransport`를 통해 애플리케이션에 등록하여 요청을 처리하게 할 수 있다.

2. Swagger UI를 통한 문서 시각화 호스팅

어떤 방법을 통해 OpenAPI 정의를 생성했든 최종적으로 이 정의를 Swagger UI를 통해 대화형 문서 형태로 시각화하여 제공해야 한다. Swagger UI는 OpenAPI 문서를 읽어 API의 모든 경로를 브라우저에 상호작용 가능한 형태로 렌더링하며, 사용자가 매개변수를 입력하고 요청을 실행할 수 있는 기능을 포함한다.

Vapor 애플리케이션에서 이러한 문서를 호스팅하기 위해 3가지 설정이 필요하다

[정적 파일 호스팅 준비]

생성된 OpenAPI 정의 파일을 애플리케이션의 `Public/` 디렉터리에 위치시켜야 한다

[미들웨어 설정]

`main.swift` 파일 내에서 `FileMiddleware`를 사용하여 `Public/` 디렉터리의 콘텐츠를 서빙하도록 Vapor 애플리케이션에 등록해야 한다. 이 때 프로세스가 파일을 정확히 찾을 수 있도록 Xcode 스킴 설정에서 사용자가 정의 작업 디렉터리를 `Package.swift`가 포함된 경로로 설정하는 것이 중요하다

[문서 접근]

설정이 완료되면

<http://localhost:8080/openapi.yaml> 같은 경로를 통해 해당 문서를 텍스트 형태로 접근할 수 있으며 시각화된 문서를 제공 받게 된다.

OpenAPI 정의 자체에는 API 호스팅 위치를 명확히 하기 위해 `Servers` 필드가 포함되어야 한다. 이 필드가 정확히 정의되어야만 Swagger UI의 기능이 실제로 해당 서버 URL을 기반으로 API 호출을 시도할 수 있다. 인증이 필요한 컨트롤러 기능을 문서화하기 위해서는 API 키와 같은 `security Schemes`를 전역으로 정의하고 해당 작업 정의 내에 `security` 필드를 통해 이를 참조함으로써 클라이언트가 필요한 인증 요구 사항을 시각적으로

확인하고 토큰 값을 입력하여 테스트할 수 있게 된다.

아래 코드는 컨트롤러에 스웨거 기능을 넣어 완성된 Swagger UI를 보여준다.

```
import Vapor
import Fluent
import VaporToOpenAPI
struct PatientController: RouteCollection {
    func boot(routes: any RoutesBuilder) throws {
        let patients = routes.grouped("api",
"patients")
        let protected =
patients.grouped(JWTMiddleware())

        protected.get("all", use: list)
            .openAPI(
                tags: TagObject(name:
TagObjectValue.patient),
                summary: "환자 전체 조회",
                description: "병원의 모든 환자
목록을 조회합니다.",
                response: .type(CommonResponseDTO<[PatientD
etailResponse]>.self)
            )
        protected.get("ward", ":ward", use:
getByWard)
            .openAPI(
                tags: TagObject(name:
TagObjectValue.patient),
                summary: "병동별 환자 조회",
                description: "특정 병동의 환자
목록을 조회합니다.",
                response: .type(CommonResponseDTO<[PatientDT
```

```
O]>.self)
            )
        protected.get(":id", use: get)
            .openAPI(
                tags: TagObject(name:
TagObjectValue.patient),
                summary: "환자 상세 조회",
                description: "특정 병동의 환자
목록을 조회합니다. 디바이스 연결 정보 및 현재
위치 포함",
                response: .type(CommonResponseDTO<PatientDe
tailResponse>.self)
            )
        protected.post("create", use: create)
            .openAPI(
                tags: TagObject(name:
TagObjectValue.patient),
                summary: "환자 등록",
                description: "새로운 환자를 등
록합니다. 디바이스 시리얼 넘버를 통해 디바스와
연결 할 수 있다.",
                body: .type(PatientAddRequest.self),
                response: .type(CommonResponseDTO<PatientDT
O>.self)
            )
        protected.patch(":id", use: update)
            .openAPI(
                tags: TagObject(name:
TagObjectValue.patient),
                summary: "환자 정보 수정",
                description: "기존 환자의 정보
를 수정합니다. 병동, 병상, 부서, 메모 등을 업데
이트할 수 있습니다.",
```

```

body: .type(PatientUpdateRequest.self),

response: .type(CommonResponseDTO<PatientDT
O>.self)
    )
    protected.delete(":id", use: delete)
    .openAPI(
        tags: TagObject(name:
TagObjectValue.patient),
        summary: "환자 삭제",
        description: "환자 정보를 삭제
합니다. 연결된 디바이스는 자동으로 해제됩니다",

response: .type(CommonResponseDTO<EmptyRes
ponse>.self)
    )
}

func list(_ req: Request) async throws ->
CommonResponseDTO<[PatientDetailResponse]>
{
    let sessionToken = try req.requireAuth()
    let service =
req.di.makePatientService(request: req)
    let patients = try await
service.getAllPatients(hospitalId:
sessionToken.hospitalId)
    let result = patients.map
{ PatientDetailResponse(from: $0) }

    return
CommonResponseDTO.success(code:
ResponseCode.COMMON200, message: "환자 목
록 조회 성공", result: result)
}

```

```

func getByWard(_ req: Request) async throws
-> CommonResponseDTO<[PatientDTO]> {
    let sessionToken = try req.requireAuth()
    let ward = try
req.parameters.require("ward", as: String.self)
    let service =
req.di.makePatientService(request: req)
    let patients = try await
service.getPatientsByWard(ward: ward, hospitalId:
sessionToken.hospitalId)
    let result = patients.map
{ PatientDTO(from: $0) }

    return
CommonResponseDTO.success(code:
ResponseCode.COMMON200, message: "병동 별
환자 조회 성공", result: result)
}

func get(_ req: Request) async throws ->
CommonResponseDTO<PatientDetailResponse> {
    let sessionToken = try req.requireAuth()
    let id = try req.parameters.require("id", as:
UUID.self)
    let service =
req.di.makePatientService(request: req)
    let patient = try await
service.getPatient(id: id, hospitalId:
sessionToken.hospitalId)
    let result = PatientDetailResponse(from:
patient)

    return
CommonResponseDTO.success(code:
ResponseCode.COMMON200, message: "환자 상

```

```

세 조회 성공", result: result)
    }

    func create(_ req: Request) async throws ->
CommonResponseDTO<PatientDTO> {
        let dto = try
req.content.decode(PatientAddRequest.self)
        let sessionToken = try req.requireAuth()
        let service =
req.di.makePatientService(request: req)

        let patient = try await
service.createPatient(
            hospitalId: sessionToken.hospitalId,
            name: dto.name,
            ward: dto.ward,
            bed: dto.bed,
            departmentId: dto.departmentId,
            deviceSerial: dto.deviceSerial,
            memo: dto.memo
        )

        try await patient.$device.load(on: req.db)
        try await patient.$department.load(on:
req.db)

        let result = PatientDTO(from: patient)
        return
CommonResponseDTO.success(code:
ResponseCode.CREATED201, message: "환자 등록
성공", result: result)
    }

    func update(_ req: Request) async throws ->
CommonResponseDTO<PatientDTO> {
        let dto = try

```

```

req.content.decode(PatientUpdateRequest.self)
        let sessionToken = try req.requireAuth()
        let id = try req.parameters.require("id", as:
UUID.self)

        let service =
req.di.makePatientService(request: req)

        let patient = try await
service.updatePatient(
            id: id,
            hospitalId: sessionToken.hospitalId,
            name: dto.name,
            ward: dto.ward,
            bed: dto.bed,
            departmentId: dto.departmentId,
            memo: dto.memo
        )

        try await patient.$device.load(on: req.db)
        try await patient.$department.load(on:
req.db)

        let result = PatientDTO(from: patient)
        return
CommonResponseDTO.success(code:
ResponseCode.COMMON200, message: "환자 수
정 성공", result: result)
    }

    func delete(_ req: Request) async throws ->
CommonResponseDTO<EmptyResponse> {
        let sessionToken = try req.requireAuth()
        let id = try req.parameters.require("id", as:
UUID.self)

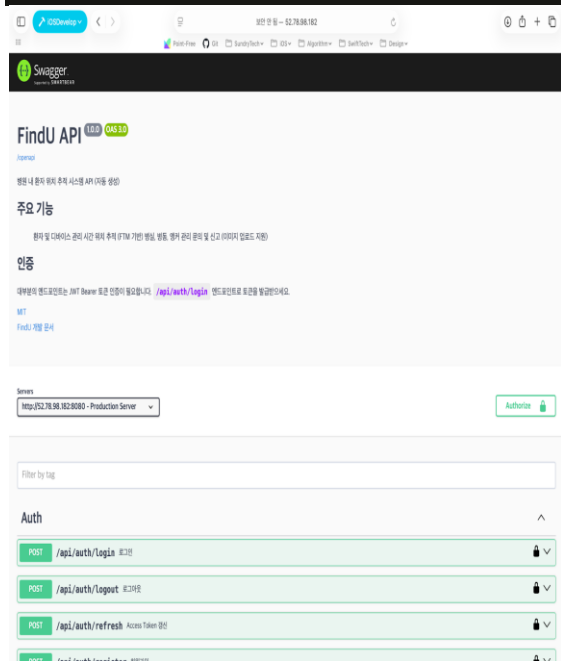
        let service =

```

```
req.di.makePatientService(request: req)

    try await service.deletePatient(id: id,
hospitalId: sessionToken.hospitalId)

    return
CommonResponseDTO.successNoData(code:
ResponseCode.COMMON200, message: "환자 삭제 성공")
}
}
```



결론

Apple의 SwiftNIO 기반 위에서 구축된 Vapor 프레임워크를 활용하여, 강력하며 타입 안정성이 보장되는 서버 애플리케이션을 개발하는 전 과정을 체계적으로 작성되었다.

개발 과정은 우선 Fluent ORM과 그 속성 래퍼를 활용하여 복잡한 엔티티 간 관계를 모델링하고, 데이터 영속성을 책임지는 구조를 정교하게 설계하는 단계에서 출발했다.

이후 요청에 대한 사전 검증 및 공통 로직을 처리하는 Middleware를 구성하고, 의존성 주입을 통한 모듈 간 결합도 감소 및 재사용성 향상을 위해 Service 계층과 DI(Container) 구조를 구축했다. 마지막으로, 핵심 비즈니스 로직을 실행하고 외부 요청과 응답을 매개하는 Controller 계층을 통해 Vapor의 견고한 Request Processing Pipeline을 완성하였다.

이 모든 흐름은 Swift 언어를 기반으로 한 팀 개발 과정에서 일관된 코드 스타일과 높은 안정성을 유지하도록 도와주었고, 실제로 팀이 해결해야 했던 요구사항들을 Vapor 기반 백엔드 아키텍처 안에서 온전히 구현하고 마무리할 수 있는 기반이 되었다.

IMDF_Indoor Map



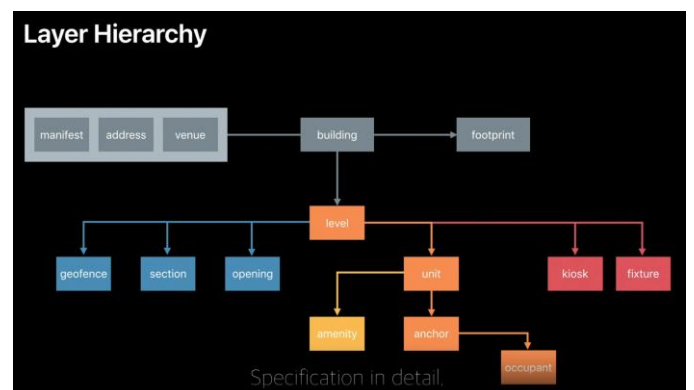
IMDF의 개요와 목적

IMDF는 애플이 설계하고 Open Geospatial Consortium에서 채택한, 실내 지도를 표현하기 위한 데이터 포맷이다. 기술적으로는 GeoJSON을 기반으로 하고 있으며, 실내 공간의 구조와 기능적 요소(층, 상점, 출입구, 편의시설 등)를 표준화된 방식으로 모델링한다. IMDF의 기본 목적은 한번 제작한 실내 지도를 모바일 및 웹 환경에서 손쉽게 재사용하고, 다양한 플랫폼과 서비스에서 일관되게 활용할 수 있게 하는 데 있다.

IMDF가 GeoJSON을 기반으로 설계된 이유는 GeoJSON의 경량성, 범용성 그리고 지도 API 및 GIS 도구와의 높은 호환성 때문이다. 또한 IMDF는 모바일 기기에서의 성능을 고려하여 설계되어 데이터 크기가 비교적 작고, 클라이언트에서 빠르게 파싱하여 렌더링할 수 있도록 구성된다.

전체 구조 개관

IMDF 패키지는 여러 개의 GeoJSON 파일과 하나의 manifest.json으로 구성된 디렉터리 구조를 가지며, 최종적으로는 하나의 압축 패키지로 묶여 서비스 제공자 또는 플랫폼에 업로드된다. 각 파일은 특정한 실내 데이터 계층을 담당하며, 이들 파일이 서로 참조 관계를 통해 완전한 실내 지도를 구성한다. 모든 GeoJSON 파일은 공통적으로 id, feature_type, geometry 속성을 반드시 포함해야 한다. 여기서 id는 해당 피처를 고유하게 식별하는 문자열이며, feature_type은 해당 피처가 어떤 종류인지를 명시하는 속성이다.



주요 파일과 속성의 상세 설명

[manifest.json]

Manifest.json은 IMDF 패키지에서 단 하나뿐인 메타데이터 파일이며 패키지의 버전, 생성 시각, 데이터 언어, 제작자 연락처, 패키지 내 포함 파일 목록과 각 파일의 체크섬 또는 해시 값 등을 포함할 수 있다. 이 파일을 통해 수신 서버가 패키지의 무결성을 검증할 수 있게 해야한다.

[address.geojson]

베뉴의 표준 주소 정보가 들어온다. 이 파일은 외부 네비게이션 시스템이나 주소 검색 서비스와의 통합을 위한 용도로 사용되며, 도로명 주소, 번지 정보, 우편번호 등의 항목을 포함할 수 있다.

[venue.geojson]

전체 실내 지도의 범위와 성격을 정의한다. 이파일에는 베뉴의 이름, 설명, 운영 시간 웹 사이트, 연락처 같은 메타 정보가 포함된다.

[building.geojson]

건물별로 분리된 개념을 담고 있어 복합 물이나 캠퍼스처럼 여러 건물로 구성된 베뉴를 명확히 기술하는데 사용된다.

[level.geojson]

층을 식별하는 중요한 파일로, 각 레벨 피처는 고유한 id를 가져야 한다. 레벨에는 층을 비교하거

나 정렬하는 데 사용할 수 있는 속성이 포함될 수 있다. 속성은 다중층을 가진 건물에서 사용자가 현재 층을 식별하고 층간 전환을 처리하는 데 필수적이다.

[unit.geojson]

각 유닛의 물리적 경계와 관련 메타데이터가 포함된다. Id, name, category, level_id 등 unit은 폴리곤을 사용하며, 프로퍼티에 다음과 같은 항목들이 작성된다.



데이터 작성 원칙 및 프랙티스

IMDF 데이터를 작성할 때에는 전체 구조의 일관성과 참조 무결성을 유지하는 것이 가장 중요한 원칙이다. 모든 피처는 고유한 id 값을 가져야 하며, 이러한 식별자는 서로 다른 파일 간의 관계를 정확하게 표현하기 위한 기반이 된다. unit이 속한 층을 가리키는 level_id는 반드시 level.geojson 내부에 존재하는 유효한 값이어야 하며, opening이 연결하는 두 공간 또한 실제로 존재하는 unit이어야 한다

실내 지도의 정확도를 확보하기 위해 좌표계 역시 패키지 전체에서 일관되게 사용되어야 한다. 보통 WGS84 기반의 좌표계가 사용되며, 좌표의 정밀도는 실내 구조를 충분히 표현할 수 있는 수준으로 유지한다. 그러나 불필요하게 지나치게 정밀한 좌표를 사용하는 것은 데이터 크기를 늘리고 모바일 렌더링 성능을 저하시킬 수 있으므로 주의해야 한다

실내 공간을 모델링할 때는 디테일과 단순화의 균형이 필요하다. 건물의 주요 이동 경로, 출입구, 계단이나 엘리베이터 같이 사용자 이동에 영향을 주는 요소는 가능한 한 정확하게 표현해야 한다. 반면 단순한 벽면이나 넓은 공간의 불필요한 세부 분할은 지도를 복잡하게 만들 수 있으므로 적절한 단순화를 적용하는 것이 좋다.

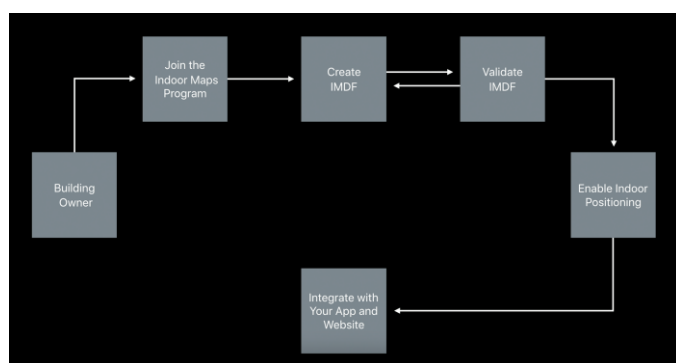
또한 분류 체계를 명확하게 유지하는 것은 검색과 필터링, 접근성 안내 등 다양한 기능의 기반이 된다. 유닛의 종류, 편의시설의 종류 등은 일관된 카테고리로 분류하여 사용자 경험의 혼란을 방지해야 한다. 접근성 요소 역시 중요한데, 휠체어 이동 가능 여부, 엘리베이터 존재 여부, 문턱의 유무 등은 필수적으로 포함하면 좋다.

각 피처의 properties에는 운영시간, 업종, 연락처, 점유자 정보 등 서비스 제공에 필요한 메타데이터를 충분히 기입하는 것이 이상적이다. 메타데이터가 충실할수록 검색 기능의 정확도가 높아지고, 사용자 인터페이스에서도 더 풍부한 정보를 제공할 수 있게 된다.

제작 및 검증 프로세스

IMDF 패키지를 제작하는 과정은 보통 설계, 데이터 수집, 디지털화, 속성 입력, 내부 검증, 포맷 검증, 패키징과 제출의 단계로 이루어진다. 먼저 설계 단계에서는 메뉴의 범위를 정의하고, 어떤 용도로 사용할 것인지에 따라 포함해야 할 데이터의 종류와 세부 속성을 결정한다. 이후 설계된 구조를 바탕으로 도면, CAD 파일, BIM 정보, 현장 조사 등을 통해 실제 실내 공간 데이터를 수집한다.

수집한 데이터를 기반으로 GeoJSON 형식의 디지털화 작업이 이루어지는데, 이 단계에서 공간의 면적, 좌표, 연결 구조 등을 정밀하게 변환한다. 속성 입력 단계에서는 각 피처에 필요한 이름, 분류, 운영 시간, 접근성 정보 등을 메타데이터로 채워 넣는다.



데이터 입력이 완료되어 내부 검증을 통해 논리적 오류나 누락된 관계, 잘못된 연결 정보 등을 점검한다. 예를 들어 계단이 잘못된 층에 연결되어 있거나 이동 경로가 실제로는 막혀 있는 경우를 확인하는 식이다. 내부 검증이 끝나면 자동화

된 포맷 검증 도구를 활용해 JSON 문법 오류, 참조 무결성, 좌표계 통일 여부 등을 검사한다.

모든 검증을 통과하면 패키지를 압축하여 제출용 파일로 준비한다.